# Slater

Version 1.0

(c) 2026 Production Labs LLC. All rights reserved.

Source Code Deposit for U.S. Copyright Registration

Total source lines: 10953 | First 1,000 + Last 1,000 lines

```
========================================================================
FILE: import.js
========================================================================
```

```
const { Pool } = require('pg');
const fs = require('fs');
require('dotenv').config();

const pool = new Pool({ connectionString: process.env.DATABASE_URL });

async function importProjects() {
  const filePath = process.argv[2];
  if (!filePath) {
 console.error('Usage: node import.js <path-to-backup.json>');
 process.exit(1);
  }

  const raw = fs.readFileSync(filePath, 'utf8');
  const backup = JSON.parse(raw);

  // Projects are nested inside backup.projects
  const projects = backup.projects || backup;
  const keys = Object.keys(projects).filter(k => k.startsWith('cs_'));

  console.log(`Importing ${keys.length} projects...`);

  for (const key of keys) {
 const project = projects[key];
 const label = project.label || key;
 try {
   await pool.query(
  `INSERT INTO projects (key, label, data, updated_at)
   VALUES ($1, $2, $3, NOW())
   ON CONFLICT (key) DO UPDATE
   SET label = $2, data = $3, updated_at = NOW()`,
  [key, label, project]
   );
   console.log(`  ${label}`);
 } catch (err) {
   console.error(`  ${key}: ${err.message}`);
 }
  }

  console.log('Done!');
  await pool.end();
}

importProjects();
```

```
========================================================================
FILE: migrate-agencies.js
========================================================================
```

```
// Run once: node migrate-agencies.js
require('dotenv').config();
const { Pool } = require('pg');
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

async function migrate() {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');

    // 1. Add dba column to users
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS dba TEXT`);

    // 2. Add billing_email and invoicing_text columns if they don't exist yet
```

```
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS agency_billing_email TEXT`);
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS agency_invoicing_text TEXT`);

    // 3. Create agencies table
    await client.query(`
      CREATE TABLE IF NOT EXISTS agencies (
        id SERIAL PRIMARY KEY,
        user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
        name TEXT,
        contact_name TEXT,
        contact_email TEXT,
        contact_phone TEXT,
        address TEXT,
        city TEXT,
        state TEXT,
        zip TEXT,
        invoicing_email TEXT,
        invoicing_text TEXT,
        default_project_type TEXT DEFAULT 'location_shoot',
        website TEXT,
        timezone TEXT DEFAULT 'PT',
        logo TEXT,
        is_default BOOLEAN DEFAULT false,
        created_at TIMESTAMPTZ DEFAULT NOW(),
        updated_at TIMESTAMPTZ DEFAULT NOW()
      )
    `);

    // 4. Add agency_id to projects
    await client.query(`ALTER TABLE projects ADD COLUMN IF NOT EXISTS agency_id INTEGER REFERENCES agenc

    // 5. Migrate existing agency data from users into agencies table (skip if already done)
    const existing = await client.query(`SELECT COUNT(*) FROM agencies`);
    if (parseInt(existing.rows[0].count) === 0) {
      await client.query(`
        INSERT INTO agencies (user_id, name, contact_name, contact_email, contact_phone,
          address, city, state, zip, invoicing_email, invoicing_text,
          default_project_type, timezone, logo, is_default)
        SELECT id, agency_name, agency_billing_contact, agency_billing_email, agency_phone,
          agency_address, agency_city, agency_state, agency_zip, agency_billing_email,
          agency_invoicing_text, agency_project_type, agency_timezone, agency_logo, true
        FROM users
        WHERE agency_name IS NOT NULL AND agency_name != ''
      `);
      console.log('Migrated agency data from users table');

      // 6. Assign default agency to existing projects
      await client.query(`
        UPDATE projects p
        SET agency_id = (
          SELECT a.id FROM agencies a WHERE a.user_id = p.owner_id AND a.is_default = true LIMIT 1
        )
        WHERE p.owner_id IS NOT NULL AND p.agency_id IS NULL
      `);
      console.log('Assigned agency_id to existing projects');
    } else {
      console.log('Agencies table already has data, skipping migration');
    }

    await client.query('COMMIT');
    console.log('Migration complete.');
  } catch (err) {
    await client.query('ROLLBACK');
    console.error('Migration failed:', err);
    process.exit(1);
  } finally {
```

```
    client.release();
    pool.end();
  }
}

migrate();
```

FILE: package.json

```
{
  "name": "slater",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "bcrypt": "^6.0.0",
    "connect-pg-simple": "^10.0.0",
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "express-rate-limit": "^8.6.1",
    "express-session": "^1.19.0",
    "pg": "^8.22.0",
    "resend": "^6.19.0"
  },
  "allowScripts": {
    "bcrypt@6.0.0": true
  }
}
```

FILE: public/admin-login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Slater Admin</title>
  <link href="https://fonts.googleapis.com/css2?family=Geist:wght@400;500;600&display=swap" rel="stylesh
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: 'Geist', sans-serif; background: #f3f0ed; display: flex; align-items: center; ju
    .card { background: #fff; border-radius: 12px; padding: 40px; width: 100%; max-width: 380px; box-sha
    .logo { display: flex; align-items: center; gap: 12px; margin-bottom: 32px; }
    .logo-text h1 { font-size: 22px; font-weight: 600; color: #222; }
    .logo-text p { font-size: 12px; color: #9D9D99; }
    label { display: block; font-size: 12px; font-weight: 500; color: #555; margin-bottom: 6px; }
    input { width: 100%; padding: 10px 12px; border: 1px solid #ddd; border-radius: 6px; font-size: 14px
    input:focus { border-color: #222; }
    button { width: 100%; padding: 11px; background: #222; color: #fff; border: none; border-radius: 6px
    button:hover { background: #333; }
    .error { color: #D94B43; font-size: 13px; margin-bottom: 16px; display: none; }
    .back-link { text-align: center; margin-top: 20px; font-size: 13px; color: #9D9D99; }
    .back-link a { color: #222; text-decoration: none; font-weight: 500; }
    .back-link a:hover { text-decoration: underline; }
  </style>
```

```
</head>
<body>
  <div class="card">
    <div class="logo">
      <div class="logo-text">
        <h1>Slater Admin</h1>
        <p>Production Labs LLC</p>
      </div>
    </div>
    <div class="error" id="error-msg"></div>
    <label>Admin Password</label>
    <input type="password" id="secret" placeholder="""""""""" autofocus>
    <button onclick="adminLogin()">Sign in</button>
    <div class="back-link"><a href="/login">Back to Slater</a></div>
  </div>
  <script>
    async function adminLogin() {
      var secret = document.getElementById('secret').value;
      var errEl = document.getElementById('error-msg');
      errEl.style.display = 'none';
      try {
        var r = await fetch('/admin/login', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ secret: secret })
        });
        var data = await r.json();
        if (!r.ok) {
          errEl.textContent = data.error || 'Login failed';
          errEl.style.display = 'block';
          return;
        }
        window.location.href = '/admin';
      } catch (e) {
        errEl.textContent = 'Connection error';
        errEl.style.display = 'block';
      }
    }

    document.addEventListener('keydown', function(e) {
      if (e.key === 'Enter') adminLogin();
    });
  </script>
</body>
</html>
```

==========

FILE: public/admin.html

==========

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Slater Admin</title>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600&display=swap" rel="stylesh
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      font-family: 'Inter', sans-serif;
      background: #111111;
      color: #F7F5EF;
      min-height: 100vh;
    }

```

```
    /* Top bar */
    .topbar {
      background: #1A1A1A;
      color: #F7F5EF;
      padding: 0 28px;
      height: 52px;
      display: flex;
      align-items: center;
      justify-content: space-between;
      position: sticky;
      top: 0;
      z-index: 100;
      border-bottom: 1px solid #2E2E2E;
    }
    .topbar-left { display: flex; align-items: baseline; gap: 12px; }
    .topbar-left h1 { font-size: 17px; font-weight: 600; letter-spacing: -0.01em; }
    .topbar-left span { font-size: 12px; color: #6E747B; }
    .topbar-right a {
      color: #6E747B;
      text-decoration: none;
      font-size: 13px;
      transition: color .15s;
    }
    .topbar-right a:hover { color: #F7F5EF; }

    /* Content wrapper */
    .content { max-width: 1100px; margin: 0 auto; padding: 28px 24px 60px; }

    /* Stats bar */
    .stats-bar {
      display: grid;
      grid-template-columns: repeat(4, 1fr);
      gap: 14px;
      margin-bottom: 28px;
    }
    .stat-box {
      background: #1A1A1A;
      border: 1px solid #2E2E2E;
      border-radius: 10px;
      padding: 20px 20px 16px;
    }
    .stat-box .stat-label { font-size: 11px; font-weight: 500; color: #6E747B; text-transform: uppercase
    .stat-box .stat-value { font-size: 32px; font-weight: 600; color: #F7F5EF; line-height: 1; }

    /* Section cards */
    .section {
      background: #1A1A1A;
      border: 1px solid #2E2E2E;
      border-radius: 10px;
      margin-bottom: 24px;
      overflow: hidden;
    }
    .section-header {
      padding: 16px 20px;
      border-bottom: 1px solid #2E2E2E;
      display: flex;
      align-items: center;
      justify-content: space-between;
    }
    .section-header h2 { font-size: 14px; font-weight: 600; color: #F7F5EF; }
    .section-body { padding: 20px; }

    /* Generate keys form */
    .gen-form {
      display: flex;
      align-items: flex-end;
```

```
      gap: 12px;
      flex-wrap: wrap;
    }
    .gen-form .field { display: flex; flex-direction: column; gap: 5px; }
    .gen-form label { font-size: 11px; font-weight: 500; color: #6E747B; text-transform: uppercase; lett
    .gen-form input, .gen-form select {
      padding: 8px 10px;
      border: 1px solid #2E2E2E;
      border-radius: 6px;
      font-size: 13px;
      font-family: inherit;
      background: #222222;
      color: #F7F5EF;
      outline: none;
      transition: border-color .15s;
    }
    .gen-form input:focus, .gen-form select:focus { border-color: #FF4D00; }
    .gen-form input[type=number] { width: 72px; }
    .gen-form input[type=date] { width: 150px; }
    .btn {
      padding: 8px 18px;
      border: none;
      border-radius: 6px;
      font-size: 13px;
      font-family: inherit;
      font-weight: 500;
      cursor: pointer;
      transition: background .15s;
    }
    .btn-primary { background: #FF4D00; color: #fff; }
    .btn-primary:hover { background: #e04000; }
    .btn-danger { background: transparent; color: #FF4D00; border: 1px solid #FF4D00; padding: 5px 10px;
    .btn-danger:hover { background: #FF4D00; color: #fff; }

    .gen-results {
      margin-top: 16px;
      display: none;
    }
    .gen-results .result-label { font-size: 12px; color: #6E747B; margin-bottom: 8px; }
    .key-list {
      display: flex;
      flex-direction: column;
      gap: 6px;
    }
    .key-item {
      font-family: 'Inter', monospace;
      font-size: 13px;
      background: #222222;
      border: 1px solid #2E2E2E;
      border-radius: 6px;
      padding: 8px 12px;
      letter-spacing: 0.08em;
      color: #F7F5EF;
      user-select: all;
    }

    /* Tables */
    .table-wrap { overflow-x: auto; }
    table {
      width: 100%;
      border-collapse: collapse;
      font-size: 13px;
    }
    thead th {
      background: #222222;
      padding: 9px 14px;
```

```
      text-align: left;
      font-size: 11px;
      font-weight: 600;
      color: #6E747B;
      text-transform: uppercase;
      letter-spacing: 0.06em;
      border-bottom: 1px solid #2E2E2E;
      white-space: nowrap;
    }
    tbody tr { border-bottom: 1px solid #2E2E2E; }
    tbody tr:last-child { border-bottom: none; }
    tbody tr:nth-child(even) { background: #1E1E1E; }
    tbody tr.expired { background: rgba(255,77,0,.07); }
    tbody tr.expired:nth-child(even) { background: rgba(255,77,0,.12); }
    tbody td { padding: 10px 14px; vertical-align: middle; color: #F7F5EF; }
    tbody td.muted { color: #6E747B; font-size: 12px; }

    .badge {
      display: inline-block;
      padding: 2px 7px;
      border-radius: 4px;
      font-size: 11px;
      font-weight: 600;
      letter-spacing: 0.04em;
    }
    .badge-beta { background: #3a2a10; color: #FFC107; }
    .badge-solo { background: #1a3a2a; color: #4CAF77; }
    .badge-team { background: #1a2a4a; color: #5B9BD5; }

    .mono { font-family: 'Inter', monospace; letter-spacing: 0.05em; font-size: 12px; }

    .empty-row td { text-align: center; color: #6E747B; padding: 28px; font-size: 13px; }

    /* Feedback banner */
    .banner {
      position: fixed;
      bottom: 24px;
      left: 50%;
      transform: translateX(-50%);
      background: #1A1A1A;
      color: #F7F5EF;
      padding: 10px 20px;
      border-radius: 8px;
      font-size: 13px;
      display: none;
      z-index: 999;
      border: 1px solid #2E2E2E;
    }
    .banner.error { background: #FF4D00; border-color: #FF4D00; }

    @media (max-width: 700px) {
      .stats-bar { grid-template-columns: repeat(2, 1fr); }
      .gen-form { flex-direction: column; align-items: stretch; }
      .gen-form input[type=number], .gen-form input[type=date] { width: 100%; }
    }
  </style>
</head>
<body>

  <!-- Top bar -->
  <div class="topbar">
    <div class="topbar-left">
      <h1>Slater Admin</h1>
      <span>Production Labs LLC</span>
    </div>
    <div class="topbar-right">
```

```
      <a href="/admin/logout">Logout</a>
    </div>
  </div>

  <div class="content">

    <!-- Stats -->
    <div class="stats-bar">
      <div class="stat-box">
        <div class="stat-label">Total Users</div>
        <div class="stat-value" id="stat-users">-</div>
      </div>
      <div class="stat-box">
        <div class="stat-label">Total Licenses</div>
        <div class="stat-value" id="stat-licenses">-</div>
      </div>
      <div class="stat-box">
        <div class="stat-label">Active Devices</div>
        <div class="stat-value" id="stat-devices">-</div>
      </div>
      <div class="stat-box">
        <div class="stat-label">Total Projects</div>
        <div class="stat-value" id="stat-projects">-</div>
      </div>
    </div>

    <!-- Generate Keys -->
    <div class="section">
      <div class="section-header">
        <h2>Generate License Keys</h2>
      </div>
      <div class="section-body">
        <div class="gen-form">
          <div class="field">
            <label>Count</label>
            <input type="number" id="gen-count" value="1" min="1" max="50">
          </div>
          <div class="field">
            <label>Plan</label>
            <select id="gen-plan">
              <option value="beta">Beta</option>
              <option value="solo">Solo</option>
              <option value="team">Team</option>
            </select>
          </div>
          <div class="field">
            <label>Max Devices</label>
            <input type="number" id="gen-devices" value="3" min="1" max="10">
          </div>
          <div class="field">
            <label>Expires (optional)</label>
            <input type="date" id="gen-expires">
          </div>
          <button class="btn btn-primary" onclick="generateKeys()">Generate</button>
        </div>
        <div class="gen-results" id="gen-results">
          <div class="result-label" id="gen-result-label"></div>
          <div class="key-list" id="key-list"></div>
        </div>
      </div>
    </div>

    <!-- Users Table -->
    <div class="section">
      <div class="section-header">
        <h2>Users</h2>
```

```
    </div>
    <div class="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Plan</th>
            <th>License Key</th>
            <th>Devices</th>
            <th>Joined</th>
            <th></th>
          </tr>
        </thead>
        <tbody id="users-tbody">
          <tr class="empty-row"><td colspan="7">Loading...</td></tr>
        </tbody>
      </table>
    </div>
  </div>

  <!-- Licenses Table -->
  <div class="section">
    <div class="section-header">
      <h2>Licenses</h2>
    </div>
    <div class="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Key</th>
            <th>Plan</th>
            <th>User</th>
            <th>Devices</th>
            <th>Expires</th>
            <th>Created</th>
            <th></th>
          </tr>
        </thead>
        <tbody id="licenses-tbody">
          <tr class="empty-row"><td colspan="7">Loading...</td></tr>
        </tbody>
      </table>
    </div>
  </div>

  <div class="section" id="dev-tools-section" style="display:none; border: 2px dashed #FF4D00;">
    <div class="section-header" style="background:rgba(255,77,0,.08);">
      <h2 style="color:#FF4D00;">Dev Tools (local only)</h2>
    </div>
    <div class="section-body">
      <p style="margin:0 0 12px; font-size:13px; color:#6E747B;">Syncs the production database down to
      <button class="btn btn-danger" onclick="syncProdDb()">Sync Production DB to Local</button>
      <pre id="sync-log" style="display:none; margin-top:16px; background:#222222; padding:12px; borde
    </div>
  </div>

</div>

<!-- Feedback banner -->
<div class="banner" id="banner"></div>

<script>
  function showBanner(msg, isError) {
    var el = document.getElementById('banner');
    el.textContent = msg;
```

```
    el.className = 'banner' + (isError ? ' error' : '');
    el.style.display = 'block';
    clearTimeout(el._timer);
    el._timer = setTimeout(function() { el.style.display = 'none'; }, 3000);
  }

  function fmtDate(iso) {
    if (!iso) return '-';
    var d = new Date(iso);
    return (d.getMonth() + 1) + '/' + d.getDate() + '/' + String(d.getFullYear()).slice(2);
  }

  function planBadge(plan) {
    if (!plan) return '<span class="muted">-</span>';
    var cls = plan === 'beta' ? 'badge-beta' : plan === 'solo' ? 'badge-solo' : 'badge-team';
    return '<span class="badge ' + cls + '">' + plan + '</span>';
  }

  function isExpired(dateStr) {
    if (!dateStr) return false;
    return new Date(dateStr) < new Date();
  }

  // Load stats
  async function loadStats() {
    try {
      var r = await fetch('/api/admin/stats');
      if (!r.ok) return;
      var d = await r.json();
      document.getElementById('stat-users').textContent = d.users;
      document.getElementById('stat-licenses').textContent = d.licenses;
      document.getElementById('stat-devices').textContent = d.devices;
      document.getElementById('stat-projects').textContent = d.projects;
    } catch (e) { /* silently skip */ }
  }

  // Load users
  async function loadUsers() {
    var tbody = document.getElementById('users-tbody');
    try {
      var r = await fetch('/api/admin/users');
      if (!r.ok) throw new Error('Failed');
      var rows = await r.json();
      if (rows.length === 0) {
        tbody.innerHTML = '<tr class="empty-row"><td colspan="7">No users found</td></tr>';
        return;
      }
      tbody.innerHTML = rows.map(function(u) {
        return '<tr>' +
          '<td>' + (u.name || '<span class="muted">-</span>') + '</td>' +
          '<td>' + u.email + '</td>' +
          '<td>' + planBadge(u.plan) + '</td>' +
          '<td class="mono">' + (u.license_key || '<span class="muted">-</span>') + '</td>' +
          '<td>' + (u.device_count || 0) + '</td>' +
          '<td class="muted">' + fmtDate(u.created_at) + '</td>' +
          '<td><button class="btn btn-danger" onclick="deleteUser(' + u.id + ', \'' + (u.email || '').
          '</tr>';
      }).join('');
    } catch (e) {
      tbody.innerHTML = '<tr class="empty-row"><td colspan="7">Error loading users</td></tr>';
    }
  }

  // Load licenses
  async function loadLicenses() {
    var tbody = document.getElementById('licenses-tbody');
```

```
    try {
      var r = await fetch('/api/admin/licenses');
      if (!r.ok) throw new Error('Failed');
      var rows = await r.json();
      if (rows.length === 0) {
        tbody.innerHTML = '<tr class="empty-row"><td colspan="7">No licenses found</td></tr>';
        return;
      }
      tbody.innerHTML = rows.map(function(l) {
        var rowClass = isExpired(l.expires_at) ? ' class="expired"' : '';
        var userCell = l.user_email
          ? (l.user_name ? l.user_name + '<br><span class="muted">' + l.user_email + '</span>' : l.use
          : '<span class="muted">Unassigned</span>';
        var devCell = (l.device_count || 0) + ' / ' + (l.max_devices || 3);
        return '<tr' + rowClass + '>' +
          '<td class="mono">' + l.key + '</td>' +
          '<td>' + planBadge(l.plan) + '</td>' +
          '<td>' + userCell + '</td>' +
          '<td>' + devCell + '</td>' +
          '<td class="muted">' + fmtDate(l.expires_at) + '</td>' +
          '<td class="muted">' + fmtDate(l.created_at) + '</td>' +
          '<td><button class="btn btn-danger" onclick="revokeLicense(' + l.id + ', \'' + l.key.replace
          '</tr>';
      }).join('');
    } catch (e) {
      tbody.innerHTML = '<tr class="empty-row"><td colspan="7">Error loading licenses</td></tr>';
    }
  }

  async function generateKeys() {
    var count = parseInt(document.getElementById('gen-count').value) || 1;
    var plan = document.getElementById('gen-plan').value;
    var max_devices = parseInt(document.getElementById('gen-devices').value) || 3;
    var expires_at = document.getElementById('gen-expires').value || null;

    try {
      var r = await fetch('/api/admin/licenses/generate', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ count: count, plan: plan, max_devices: max_devices, expires_at: expires
      });
      var data = await r.json();
      if (!r.ok) throw new Error(data.error || 'Failed');

      var resultEl = document.getElementById('gen-results');
      var labelEl = document.getElementById('gen-result-label');
      var listEl = document.getElementById('key-list');
      labelEl.textContent = data.keys.length + ' key' + (data.keys.length !== 1 ? 's' : '') + ' genera
      listEl.innerHTML = data.keys.map(function(k) {
        return '<div class="key-item">' + k + '</div>';
      }).join('');
      resultEl.style.display = 'block';
      showBanner('Generated ' + data.keys.length + ' key' + (data.keys.length !== 1 ? 's' : ''));
      loadLicenses();
      loadStats();
    } catch (e) {
      showBanner(e.message || 'Error generating keys', true);
    }
  }

  async function deleteUser(id, email) {
    if (!confirm('Delete user "' + email + '"? This cannot be undone.')) return;
    try {
      var r = await fetch('/api/admin/users/' + id, { method: 'DELETE' });
      if (!r.ok) throw new Error('Failed');
      showBanner('User deleted');
```

```
        loadUsers();
        loadStats();
      } catch (e) {
        showBanner('Error deleting user', true);
      }
    }

    async function revokeLicense(id, key) {
      if (!confirm('Revoke license ' + key + '? This cannot be undone.')) return;
      try {
        var r = await fetch('/api/admin/licenses/' + id, { method: 'DELETE' });
        if (!r.ok) throw new Error('Failed');
        showBanner('License revoked');
        loadLicenses();
        loadUsers();
        loadStats();
      } catch (e) {
        showBanner('Error revoking license', true);
      }
    }

    async function syncProdDb() {
      if (!confirm('This will WIPE your local slater_dev database and replace it with production data. C
      var logEl = document.getElementById('sync-log');
      logEl.style.display = 'block';
      logEl.textContent = 'Running sync...';
      try {
        var r = await fetch('/api/admin/sync-prod-db', { method: 'POST' });
        var text = await r.text();
        var data;
        try { data = JSON.parse(text); } catch (_) {
          throw new Error('Server returned unexpected response (HTTP ' + r.status + '):\n' + text.slice(
        }
        if (!r.ok) throw new Error(data.error || 'Failed');
        logEl.textContent = data.log;
        showBanner('Sync complete');
      } catch (e) {
        logEl.textContent = 'ERROR: ' + e.message;
        showBanner('Sync failed', true);
      }
    }

    // Init
    loadStats();
    loadUsers();
    loadLicenses();
    if (window.location.hostname === 'localhost' || window.location.hostname === '127.0.0.1') {
      document.getElementById('dev-tools-section').style.display = 'block';
    }
  </script>
</body>
</html>
```

==========

FILE: public/app.js

==========

```
 //    Server API client
 const API = {
   base: window.location.origin,
   async getProjects() {
     try {
       const r = await fetch(this.base+'/api/projects');
       if (!r.ok) return null;
       return await r.json();
     } catch(e) { return null; }
```

```
  },
  async getProject(key) {
    try {
      const r = await fetch(this.base+'/api/projects/'+key);
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async saveProject(key, label, data) {
    try {
      const r = await fetch(this.base+'/api/projects/'+key, {
        method:'POST',
        headers:{'Content-Type':'application/json'},
        body:JSON.stringify({label, data})
      });
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async deleteProject(key) {
    try {
      const r = await fetch(this.base+'/api/projects/'+key, {method:'DELETE'});
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async saveReceipt(projectKey, index, imageData) {
    try {
      const r = await fetch(this.base+'/api/receipts/'+projectKey+'/'+index, {
        method:'POST',
        headers:{'Content-Type':'application/json'},
        body:JSON.stringify({image_data: imageData})
      });
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async getReceipts(projectKey) {
    try {
      const r = await fetch(this.base+'/api/receipts/'+projectKey);
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async deleteReceipts(projectKey) {
    try {
      const r = await fetch(this.base+'/api/receipts/'+projectKey, {method:'DELETE'});
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async getMe() {
    try {
      const r = await fetch(this.base+'/api/users/me');
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async saveMe(data) {
    try {
      const r = await fetch(this.base+'/api/users/me', {
        method:'PUT', headers:{'Content-Type':'application/json'}, body:JSON.stringify(data)
      });
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
```

```
  },
  async getAgencies() {
    try {
      const r = await fetch(this.base+'/api/agencies');
      if (!r.ok) return [];
      return await r.json();
    } catch(e) { return []; }
  },
  async createAgency(data) {
    try {
      const r = await fetch(this.base+'/api/agencies', {
        method:'POST', headers:{'Content-Type':'application/json'}, body:JSON.stringify(data)
      });
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async updateAgency(id, data) {
    try {
      const r = await fetch(this.base+'/api/agencies/'+id, {
        method:'PUT', headers:{'Content-Type':'application/json'}, body:JSON.stringify(data)
      });
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async deleteAgency(id) {
    try {
      const r = await fetch(this.base+'/api/agencies/'+id, {method:'DELETE'});
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async setDefaultAgency(id) {
    try {
      const r = await fetch(this.base+'/api/agencies/'+id+'/set-default', {method:'POST'});
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async getAgencySettings() {
    try {
      const r = await fetch(this.base+'/api/users/me/agency');
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async saveAgencySettings(data) {
    try {
      const r = await fetch(this.base+'/api/users/me/agency', {
        method:'PUT',
        headers:{'Content-Type':'application/json'},
        body:JSON.stringify(data)
      });
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async getContacts() {
    try {
      const r = await fetch(this.base+'/api/users/me/contacts');
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
  async saveContacts(data) {
```

```
    try {
      const r = await fetch(this.base+'/api/users/me/contacts', {
        method:'PUT',
        headers:{'Content-Type':'application/json'},
        body:JSON.stringify(data)
      });
      if (!r.ok) return null;
      return await r.json();
    } catch(e) { return null; }
  },
};

const LB64 = "iVBORw0KGgoAAAANSUhEUgAAAPAAAADwCAYAAAA+VemSAAEAAElEQVR42uz9d5jl11nni37WWnnnnnni37W+uXfxfrt2uXfxfrt2uXfxfrt2
```

```
    clone.id = "mobile-lib-select";
    clone.style.cssText = "width:100%;padding:8px 10px;border:1px solid var(--border);border-radius:6px;
    clone.onchange = function() { sel.value = this.value; sel.dispatchEvent(new Event("change")); closeM
    mobileSelectWrap.appendChild(clone);
  }

  var btnWrap = document.getElementById("mobile-lib-buttons");
  btnWrap.innerHTML = "";
  var actions = [
    { label: "+ New", fn: function() { libNew(); closeMobileDrawer(); } },
    { label: "Save", fn: function() { libSave(); closeMobileDrawer(); } },
    { label: "Duplicate", fn: function() { libDuplicate(); closeMobileDrawer(); } },
    { label: "Delete", fn: function() { libDelete(); closeMobileDrawer(); } },
  ];
  actions.forEach(function(a) {
    var btn = document.createElement("button");
    btn.textContent = a.label;
    btn.onclick = a.fn;
    btn.style.cssText = "width:100%;padding:10px;border:1px solid var(--border);border-radius:6px;backgr
    btnWrap.appendChild(btn);
  });

  document.getElementById("mobile-drawer").style.display = "block";
  document.body.style.overflow = "hidden";
}

function closeMobileDrawer() {
  document.getElementById("mobile-drawer").style.display = "none";
  document.body.style.overflow = "";
}

function mobileNavGo(tabId) {
  st(tabId);
  closeMobileMoreMenu();
```

==========

*** FIRST PORTION ENDS HERE ***
Lines 1-1000 of 10953 total code lines shown above.

OMITTED: lines 1001 through 9953
(8953 lines omitted per Copyright Office deposit rules)

*** LAST PORTION BEGINS HERE ***
Last 1000 of 10953 total code lines shown below.

==========

```
body.sidebar-open{padding-right:0!important;transition:none!important}

/* App shell mobile overrides */
@media(max-width:768px){
  .app-shell{grid-template-columns:1fr;height:auto;overflow:visible}
  .app-sidebar{display:none}
  .app-main{height:auto;overflow:visible}
  .app-body{flex-direction:column;overflow:visible;height:auto}
  .app-content-col{overflow:visible}
  .app-content{overflow-y:visible;max-width:100%;padding:8px 8px 80px;margin:0}
  .app-topbar{display:flex!important;overflow-x:hidden}
  .project-bar{display:none!important}
  #daily-sidebar{width:100%!important;max-width:100%!important;height:auto!important;position:relative;b
  #daily-sidebar.hidden{width:0!important;border-top-width:0}
}
```

==========

FILE: routes/agencies.js

==========

```
const express = require('express');
```

```
const router = express.Router();
const { Pool } = require('pg');
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

const FIELDS = 'id, user_id, name, contact_name, contact_email, contact_phone, address, city, state, zip

// Get all agencies for current user
router.get('/', async (req, res) => {
  try {
    const result = await pool.query(
      `SELECT ${FIELDS} FROM agencies WHERE user_id = $1 ORDER BY is_default DESC, name ASC`,
      [req.session.userId]
    );
    res.json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to load agencies' });
  }
});

// Get single agency
router.get('/:id', async (req, res) => {
  try {
    const result = await pool.query(
      `SELECT ${FIELDS} FROM agencies WHERE id = $1 AND user_id = $2`,
      [req.params.id, req.session.userId]
    );
    if (!result.rows.length) return res.status(404).json({ error: 'Not found' });
    res.json(result.rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to load agency' });
  }
});

// Create agency
router.post('/', async (req, res) => {
  try {
    const { name, contact_name, contact_email, contact_phone, address, city, state,
            zip, invoicing_email, invoicing_text, default_project_type, website,
            timezone, logo, is_default } = req.body;
    if (is_default) {
      await pool.query('UPDATE agencies SET is_default = false WHERE user_id = $1', [req.session.userId]
    }
    const result = await pool.query(
      `INSERT INTO agencies (user_id, name, contact_name, contact_email, contact_phone,
       address, city, state, zip, invoicing_email, invoicing_text, default_project_type,
       website, timezone, logo, is_default)
       VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15,$16) RETURNING ${FIELDS}`,
      [req.session.userId, name||'', contact_name||'', contact_email||'', contact_phone||'',
       address||'', city||'', state||'', zip||'', invoicing_email||'', invoicing_text||'',
       default_project_type||'location_shoot', website||'', timezone||'PT', logo||'',
       is_default || false]
    );
    res.json(result.rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to create agency' });
  }
});

// Update agency
router.put('/:id', async (req, res) => {
  try {
    const { name, contact_name, contact_email, contact_phone, address, city, state,
            zip, invoicing_email, invoicing_text, default_project_type, website,
```

```
            timezone, logo, is_default } = req.body;
    if (is_default) {
      await pool.query('UPDATE agencies SET is_default = false WHERE user_id = $1', [req.session.userId]
    }
    const result = await pool.query(
      `UPDATE agencies SET name=$1, contact_name=$2, contact_email=$3, contact_phone=$4,
       address=$5, city=$6, state=$7, zip=$8, invoicing_email=$9, invoicing_text=$10,
       default_project_type=$11, website=$12, timezone=$13, logo=$14, is_default=$15,
       updated_at=NOW() WHERE id=$16 AND user_id=$17 RETURNING ${FIELDS}`,
      [name||'', contact_name||'', contact_email||'', contact_phone||'', address||'',
       city||'', state||'', zip||'', invoicing_email||'', invoicing_text||'',
       default_project_type||'location_shoot', website||'', timezone||'PT', logo||'',
       is_default||false, req.params.id, req.session.userId]
    );
    if (!result.rows.length) return res.status(404).json({ error: 'Not found' });
    res.json(result.rows[0]);
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to update agency' });
  }
});

// Delete agency
router.delete('/:id', async (req, res) => {
  try {
    await pool.query('DELETE FROM agencies WHERE id = $1 AND user_id = $2',
      [req.params.id, req.session.userId]);
    res.json({ success: true });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to delete agency' });
  }
});

// Set default agency
router.post('/:id/set-default', async (req, res) => {
  try {
    await pool.query('UPDATE agencies SET is_default = false WHERE user_id = $1', [req.session.userId]);
    await pool.query('UPDATE agencies SET is_default = true WHERE id = $1 AND user_id = $2',
      [req.params.id, req.session.userId]);
    res.json({ success: true });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to set default' });
  }
});

module.exports = router;
```

```
========================================================================
FILE: routes/contacts.js
========================================================================
```

```
const express = require('express');
const router = express.Router();
const { Pool } = require('pg');
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

router.get('/', async (req, res) => {
  try {
    const result = await pool.query('SELECT contacts FROM users WHERE id = $1', [req.session.userId]);
    res.json(result.rows[0]?.contacts || {});
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to load contacts' });
  }
});
```

```

router.put('/', async (req, res) => {
  try {
    await pool.query('UPDATE users SET contacts = $1::jsonb, updated_at = NOW() WHERE id = $2', [JSON.st
    res.json({ success: true });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to save contacts' });
  }
});

module.exports = router;
```

```
================================================================================
FILE: routes/licenses.js
================================================================================
```

```
const express = require('express');
const router = express.Router();
const { Pool } = require('pg');
const crypto = require('crypto');
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

function generateKey() {
  const seg = () => crypto.randomBytes(2).toString('hex').toUpperCase();
  return `SLTR-${seg()}${seg()}-${seg()}${seg()}-${seg()}${seg()}`;
}

function getFingerprint(req) {
  const data = [req.headers['user-agent']||'', req.headers['accept-language']||'', req.ip||''].join('|')
  return crypto.createHash('sha256').update(data).digest('hex');
}

// Validate license key and register device
router.post('/validate', async (req, res) => {
  const { key, fingerprint } = req.body;
  if (!key) return res.status(400).json({ error: 'License key required' });
  try {
    const result = await pool.query('SELECT * FROM licenses WHERE key = $1', [key.trim().toUpperCase()])
    if (!result.rows.length) return res.status(404).json({ error: 'Invalid license key' });
    const license = result.rows[0];
    if (license.expires_at && new Date(license.expires_at) < new Date()) {
      return res.status(403).json({ error: 'License key has expired' });
    }
    const deviceResult = await pool.query(
      'SELECT * FROM license_devices WHERE license_id = $1 AND device_fingerprint = $2',
      [license.id, fingerprint]
    );
    if (deviceResult.rows.length) {
      await pool.query('UPDATE license_devices SET last_seen = NOW() WHERE license_id = $1 AND device_fi
      return res.json({ valid: true, plan: license.plan, license_id: license.id });
    }
    const countResult = await pool.query('SELECT COUNT(*) FROM license_devices WHERE license_id = $1', [
    if (parseInt(countResult.rows[0].count) >= license.max_devices) {
      return res.status(403).json({ error: `Device limit reached. This license allows ${license.max_devi
    }
    await pool.query('INSERT INTO license_devices (license_id, device_fingerprint, user_agent) VALUES ($
    res.json({ valid: true, plan: license.plan, license_id: license.id });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'License validation failed' });
  }
});

// Generate beta keys (admin only)
router.post('/generate-beta', async (req, res) => {
  const { count = 1, expires_at, secret } = req.body;
```

```
  if (secret !== process.env.ADMIN_SECRET) return res.status(403).json({ error: 'Unauthorized' });
  try {
    const keys = [];
    for (let i = 0; i < Math.min(count, 50); i++) {
      const key = generateKey();
      const result = await pool.query(
        'INSERT INTO licenses (key, plan, max_devices, expires_at) VALUES ($1, $2, $3, $4) RETURNING key
        [key, 'beta', 3, expires_at || null]
      );
      keys.push(result.rows[0].key);
    }
    res.json({ keys });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Key generation failed' });
  }
});

// Get devices for current user
router.get('/devices', async (req, res) => {
  if (!req.session.userId) return res.status(401).json({ error: 'Not logged in' });
  try {
    const userResult = await pool.query('SELECT license_id FROM users WHERE id = $1', [req.session.userI
    if (!userResult.rows[0]?.license_id) return res.json([]);
    const devices = await pool.query(
      'SELECT id, device_fingerprint, user_agent, last_seen, created_at FROM license_devices WHERE licen
      [userResult.rows[0].license_id]
    );
    res.json(devices.rows);
  } catch (err) {
    res.status(500).json({ error: 'Failed to load devices' });
  }
});

// Deactivate a device
router.delete('/devices/:id', async (req, res) => {
  if (!req.session.userId) return res.status(401).json({ error: 'Not logged in' });
  try {
    const userResult = await pool.query('SELECT license_id FROM users WHERE id = $1', [req.session.userI
    if (!userResult.rows[0]?.license_id) return res.status(403).json({ error: 'No license' });
    await pool.query('DELETE FROM license_devices WHERE id = $1 AND license_id = $2', [req.params.id, us
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: 'Failed to deactivate device' });
  }
});

module.exports = router;
```

```
========================================================================
FILE: routes/ocr.js
========================================================================
const express = require('express');
const router = express.Router();

router.post('/', async (req, res) => {
  const { image_data } = req.body;
  if (!image_data) return res.status(400).json({ error: 'No image provided' });

  try {
 const base64 = image_data.split(',')[1];
 const mediaType = image_data.split(';')[0].split(':')[1];
 console.log('Calling Anthropic, mediaType:', mediaType, 'base64 length:', base64 ? base64.length : 0);

 const response = await fetch('https://api.anthropic.com/v1/messages', {
   method: 'POST',
```

```
   headers: {
  'Content-Type': 'application/json',
  'x-api-key': process.env.ANTHROPIC_API_KEY,
  'anthropic-version': '2023-06-01',
   },
   body: JSON.stringify({
  model: 'claude-haiku-4-5-20251001',
  max_tokens: 256,
  messages: [{
    role: 'user',
    content: [
   {
     type: 'image',
     source: { type: 'base64', media_type: mediaType, data: base64 }
   },
   {
     type: 'text',
     text: 'Look carefully at this receipt for any date printed on it. Extract the following and respond
   }
    ]
  }]
   })
 });

 console.log('Anthropic status:', response.status);
 const data = await response.json();
 console.log('OCR response:', JSON.stringify(data).slice(0, 300));

 // Handle API errors
 if (data.type === 'error') {
   console.warn('Anthropic error:', data.error.message);
   return res.status(402).json({ error: data.error.message });
 }

 const text = data.content[0].text.trim().replace(/^```json\s*/,'').replace(/\s*```$/,'');
 const parsed = JSON.parse(text);
 res.json(parsed);
 } catch (err) {
  console.error('OCR error full:', err.message, err.stack);
  res.status(500).json({ error: 'OCR failed', detail: err.message });
   }
});

module.exports = router;
```

```
========================================================================
FILE: routes/projects.js
========================================================================
```

```
const express = require('express');
const router = express.Router();
const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL
});

// Get all projects (just keys and labels for the dropdown)
router.get('/', async (req, res) => {
  try {
 const result = await pool.query(
   'SELECT key, label, updated_at FROM projects WHERE owner_id = $1 ORDER BY updated_at DESC',
   [req.session.userId]
 );
 res.json(result.rows);
  } catch (err) {
 console.error(err);
```

```
 res.status(500).json({ error: 'Failed to load projects' });
  }
});

// Get a single project
router.get('/:key', async (req, res) => {
  try {
 const result = await pool.query(
   'SELECT * FROM projects WHERE key = $1 AND owner_id = $2',
   [req.params.key, req.session.userId]
 );
 if (!result.rows.length) return res.status(404).json({ error: 'Not found' });
 res.json(result.rows[0]);
  } catch (err) {
 console.error(err);
 res.status(500).json({ error: 'Failed to load project' });
  }
});

// Save a project (create or update)
router.post('/:key', async (req, res) => {
  const { key } = req.params;
  const { label, data } = req.body;
  try {
 const result = await pool.query(
   `INSERT INTO projects (key, label, data, owner_id, updated_at)
    VALUES ($1, $2, $3, $4, NOW())
    ON CONFLICT (key) DO UPDATE
    SET label = $2, data = $3, updated_at = NOW()
    WHERE projects.owner_id = $4
    RETURNING *`,
   [key, label, data, req.session.userId]
 );
 res.json(result.rows[0]);
  } catch (err) {
 console.error(err);
 res.status(500).json({ error: 'Failed to save project' });
  }
});

// Delete a project
router.delete('/:key', async (req, res) => {
  try {
 await pool.query(
   'DELETE FROM projects WHERE key = $1 AND owner_id = $2',
   [req.params.key, req.session.userId]
 );
 res.json({ success: true });
  } catch (err) {
 console.error(err);
 res.status(500).json({ error: 'Failed to delete project' });
  }
});

module.exports = router;
```

```
========================================================================
FILE: routes/receipts.js
========================================================================
```

```
const express = require('express');
const router = express.Router();
const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL
});
```

```
10346
10347 // Save a receipt
10348 router.post('/:projectKey/:index', async (req, res) => {
10349   const { projectKey, index } = req.params;
10350   const { image_data } = req.body;
10351   try {
10352  const result = await pool.query(
10353    `INSERT INTO receipts (project_key, expense_index, image_data, owner_id)
10354     VALUES ($1, $2, $3, $4)
10355     ON CONFLICT (project_key, expense_index) DO UPDATE
10356     SET image_data = $3, created_at = NOW()
10357     WHERE receipts.owner_id = $4
10358     RETURNING id`,
10359    [projectKey, parseInt(index), image_data, req.session.userId]
10360  );
10361  res.json({ success: true, id: result.rows[0].id });
10362   } catch (err) {
10363  console.error(err);
10364  res.status(500).json({ error: 'Failed to save receipt' });
10365   }
10366 });
10367
10368 // Get all receipts for a project
10369 router.get('/:projectKey', async (req, res) => {
10370   try {
10371  const result = await pool.query(
10372    'SELECT expense_index, image_data FROM receipts WHERE project_key = $1 AND owner_id = $2 ORDER BY exp
10373    [req.params.projectKey, req.session.userId]
10374  );
10375  res.json(result.rows);
10376   } catch (err) {
10377  console.error(err);
10378  res.status(500).json({ error: 'Failed to load receipts' });
10379   }
10380 });
10381
10382 // Delete all receipts for a project
10383 router.delete('/:projectKey', async (req, res) => {
10384   try {
10385  await pool.query(
10386    'DELETE FROM receipts WHERE project_key = $1 AND owner_id = $2',
10387    [req.params.projectKey, req.session.userId]
10388  );
10389  res.json({ success: true });
10390   } catch (err) {
10391  console.error(err);
10392  res.status(500).json({ error: 'Failed to delete receipts' });
10393   }
10394 });
10395
10396 module.exports = router;
```

```
========================================================================
FILE: routes/users.js
========================================================================
```

```
10397 const express = require('express');
10398 const router = express.Router();
10399 const bcrypt = require('bcrypt');
10400 const crypto = require('crypto');
10401 const { Pool } = require('pg');
10402 const rateLimit = require('express-rate-limit');
10403 const { Resend } = require('resend');
10404 const resend = new Resend(process.env.RESEND_API_KEY);
10405
10406 const loginLimiter = rateLimit({
10407   windowMs: 15 * 60 * 1000,
```

```
  max: 10,
  message: { error: 'Too many login attempts. Try again in 15 minutes.' },
  standardHeaders: true,
  legacyHeaders: false
});

const pool = new Pool({ connectionString: process.env.DATABASE_URL });

// Register a new user (requires valid license key)
router.post('/register', async (req, res) => {
  const { email, password, name, invite_code, license_key } = req.body;
  if (!email || !password) return res.status(400).json({ error: 'Email and password required' });
  if (!license_key) return res.status(400).json({ error: 'License key required' });
  if (process.env.INVITE_CODE && invite_code !== process.env.INVITE_CODE) {
    return res.status(403).json({ error: 'Invalid invite code' });
  }
  try {
    // Validate license key
    const licResult = await pool.query('SELECT * FROM licenses WHERE key = $1', [license_key.trim().toUp
    if (!licResult.rows.length) return res.status(400).json({ error: 'Invalid license key' });
    const licenseData = licResult.rows[0];
    if (licenseData.expires_at && new Date(licenseData.expires_at) < new Date()) {
      return res.status(400).json({ error: 'License key has expired' });
    }

    // Build device fingerprint
    const fpRaw = [req.headers['user-agent']||'', req.headers['accept-language']||'', req.ip||''].join('
    const fingerprint = crypto.createHash('sha256').update(fpRaw).digest('hex');

    // Check if this device is already registered for this license
    const deviceCheck = await pool.query(
      'SELECT * FROM license_devices WHERE license_id = $1 AND device_fingerprint = $2',
      [licenseData.id, fingerprint]
    );
    if (!deviceCheck.rows.length) {
      const countResult = await pool.query('SELECT COUNT(*) FROM license_devices WHERE license_id = $1',
      if (parseInt(countResult.rows[0].count) >= licenseData.max_devices) {
        return res.status(403).json({
          error: 'Device limit reached. This license allows ' + licenseData.max_devices + ' devices. Dea
          device_limit: true
        });
      }
    }

    // Create the user
    const hash = await bcrypt.hash(password, 12);
    const result = await pool.query(
      'INSERT INTO users (email, password_hash, name, license_id, license_key) VALUES ($1, $2, $3, $4, $
      [email.toLowerCase().trim(), hash, name||'', licenseData.id, license_key.trim().toUpperCase()]
    );
    const user = result.rows[0];

    // Register device for this license
    if (deviceCheck.rows.length) {
      await pool.query(
        'UPDATE license_devices SET last_seen = NOW() WHERE license_id = $1 AND device_fingerprint = $2'
        [licenseData.id, fingerprint]
      );
    } else {
      await pool.query(
        'INSERT INTO license_devices (license_id, device_fingerprint, user_agent) VALUES ($1, $2, $3)',
        [licenseData.id, fingerprint, req.headers['user-agent']||'']
      );
    }

    // Mark license as activated on first use
```

```
    if (!licenseData.activated_at) {
      await pool.query('UPDATE licenses SET activated_at = NOW() WHERE id = $1', [licenseData.id]);
    }

    // Auto-login after registration
    req.session.userId = user.id;
    req.session.userEmail = user.email;
    req.session.userName = user.name;

    res.json({ success: true, user: user });
  } catch (err) {
    if (err.code === '23505') return res.status(409).json({ error: 'Email already registered' });
    console.error(err);
    res.status(500).json({ error: 'Registration failed' });
  }
});

// Login
router.post('/login', loginLimiter, async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) return res.status(400).json({ error: 'Email and password required' });
  try {
 const result = await pool.query(
   'SELECT * FROM users WHERE email = $1',
   [email.toLowerCase().trim()]
 );
 if (!result.rows.length) return res.status(401).json({ error: 'Invalid email or password' });
 const user = result.rows[0];
 const match = await bcrypt.compare(password, user.password_hash);
 if (!match) return res.status(401).json({ error: 'Invalid email or password' });
 req.session.userId = user.id;
 req.session.userEmail = user.email;
 req.session.userName = user.name;
 res.json({ success: true, user: { id: user.id, email: user.email, name: user.name } });
  } catch (err) {
 console.error(err);
 res.status(500).json({ error: 'Login failed' });
  }
});

// Logout
router.post('/logout', (req, res) => {
  req.session.destroy();
  res.json({ success: true });
});

// Get current user / My Info
router.get('/me', async (req, res) => {
  if (!req.session.userId) return res.status(401).json({ error: 'Not logged in' });
  try {
    const result = await pool.query(
      'SELECT id, email, name, dba, phone, timezone, logo FROM users WHERE id=$1',
      [req.session.userId]
    );
    const row = result.rows[0] || {};
    res.json({
      id: row.id,
      email: row.email,
      name: row.name || '',
      dba: row.dba || '',
      phone: row.phone || '',
      timezone: row.timezone || 'PT',
      avatar: row.logo || null,
    });
  } catch (err) {
    console.error(err);
```

```
    res.status(500).json({ error: 'Failed to load user info' });
  }
});

// Update My Info
router.put('/me', async (req, res) => {
  if (!req.session.userId) return res.status(401).json({ error: 'Not logged in' });
  const { name, dba, phone, timezone, avatar } = req.body;
  try {
    await pool.query(
      'UPDATE users SET name=$1, dba=$2, phone=$3, timezone=$4, logo=$5, updated_at=NOW() WHERE id=$6',
      [name||'', dba||'', phone||'', timezone||'PT', avatar||null, req.session.userId]
    );
    req.session.userName = name || req.session.userName;
    res.json({ success: true });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to save user info' });
  }
});

// Update agency settings (legacy   kept for backwards compatibility)
router.put('/me/agency', async (req, res) => {
  if (!req.session.userId) return res.status(401).json({ error: 'Not logged in' });
  const { agency_name, agency_address, agency_city, agency_state, agency_zip,
    agency_phone, agency_billing_contact, agency_timezone, agency_project_type, agency_logo,
    agency_billing_email, agency_invoicing_text } = req.body;
  try {
 await pool.query(
    `UPDATE users SET agency_name=$1, agency_address=$2, agency_city=$3, agency_state=$4,
    agency_zip=$5, agency_phone=$6, agency_billing_contact=$7, agency_timezone=$8,
    agency_project_type=$9, agency_logo=$10, agency_billing_email=$11, agency_invoicing_text=$12,
    updated_at=NOW() WHERE id=$13`,
   [agency_name, agency_address, agency_city, agency_state, agency_zip,
    agency_phone, agency_billing_contact, agency_timezone, agency_project_type, agency_logo,
    agency_billing_email, agency_invoicing_text,
    req.session.userId]
 );
 res.json({ success: true });
  } catch (err) {
 console.error(err);
 res.status(500).json({ error: 'Failed to save agency settings' });
  }
});

// Get agency settings
router.get('/me/agency', async (req, res) => {
  if (!req.session.userId) return res.status(401).json({ error: 'Not logged in' });
  try {
 const result = await pool.query(
    `SELECT agency_name, agency_address, agency_city, agency_state, agency_zip,
    agency_phone, agency_billing_contact, agency_timezone, agency_project_type, agency_logo,
    agency_billing_email, agency_invoicing_text
    FROM users WHERE id=$1`,
   [req.session.userId]
 );
 res.json(result.rows[0] || {});
  } catch (err) {
 console.error(err);
 res.status(500).json({ error: 'Failed to load agency settings' });
  }
});

// Get contacts
router.get('/me/contacts', async (req, res) => {
  if (!req.session.userId) return res.status(401).json({ error: 'Not logged in' });
```

```
  try {
    const result = await pool.query('SELECT contacts FROM users WHERE id=$1', [req.session.userId]);
    res.json(result.rows[0]?.contacts || {});
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to load contacts' });
  }
});

// Save contacts   replaces each bucket with the incoming data
router.put('/me/contacts', async (req, res) => {
  if (!req.session.userId) return res.status(401).json({ error: 'Not logged in' });
  const incoming = req.body;
  if (!incoming || typeof incoming !== 'object' || Array.isArray(incoming)) {
    return res.status(400).json({ error: 'Invalid contacts data' });
  }
  try {
    const result = await pool.query('SELECT contacts FROM users WHERE id=$1', [req.session.userId]);
    const existing = result.rows[0]?.contacts || {};
    const merged = Object.assign({}, existing);
    Object.keys(incoming).forEach(function(bucket) {
      const items = incoming[bucket];
      if (!Array.isArray(items)) return;
      merged[bucket] = items;
    });
    await pool.query('UPDATE users SET contacts=$1, updated_at=NOW() WHERE id=$2', [merged, req.session.
    res.json({ success: true });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: 'Failed to save contacts' });
  }
});

// POST /api/users/forgot-password
router.post('/forgot-password', async (req, res) => {
  var email = req.body.email;
  if (!email) return res.status(400).json({ error: 'Email required' });
  try {
    var result = await pool.query(
      'SELECT id, name FROM users WHERE email = $1',
      [email.toLowerCase().trim()]
    );
    // Always return success even if email not found -- security best practice
    if (!result.rows.length) return res.json({ success: true });

    var user = result.rows[0];
    var token = crypto.randomBytes(32).toString('hex');
    var expires = new Date(Date.now() + 60 * 60 * 1000); // 1 hour

    // Invalidate any existing tokens
    await pool.query(
      'UPDATE password_reset_tokens SET used_at = NOW() WHERE user_id = $1 AND used_at IS NULL',
      [user.id]
    );

    // Store new token
    await pool.query(
      'INSERT INTO password_reset_tokens (user_id, token, expires_at) VALUES ($1, $2, $3)',
      [user.id, token, expires]
    );

    var resetUrl = (process.env.APP_URL || 'http://localhost:3000') + '/reset-password?token=' + token;

    await resend.emails.send({
      from: process.env.RESEND_FROM_EMAIL,
      to: email,
```

```
      subject: 'Reset your Slater password',
      html: `
        <div style="font-family:sans-serif;max-width:480px;margin:0 auto;padding:32px 24px;background:#f
          <div style="background:#222222;padding:20px 24px;border-radius:8px 8px 0 0">
            <h1 style="color:#C8A84B;font-size:22px;margin:0;letter-spacing:0.05em">SLATER</h1>
            <p style="color:rgba(255,255,255,0.5);font-size:12px;margin:4px 0 0">The Producer's Toolbox<
          </div>
          <div style="background:#ffffff;padding:24px;border-radius:0 0 8px 8px">
            <p style="color:#222222;font-size:15px">Hi ${user.name || 'there'},</p>
            <p style="color:#555555;font-size:14px;line-height:1.6">We received a request to reset your
            <div style="text-align:center;margin:28px 0">
              <a href="${resetUrl}" style="background:#222222;color:#ffffff;padding:12px 28px;border-rad
            </div>
            <p style="color:#9D9D99;font-size:12px">This link expires in 1 hour. If you didn't request a
            <hr style="border:none;border-top:1px solid #f0ebe3;margin:20px 0">
            <p style="color:#9D9D99;font-size:11px;text-align:center">Slater - The Producer's Toolbox -
          </div>
        </div>
      `
    });

    res.json({ success: true });
  } catch (err) {
    console.error('Forgot password error:', err);
    res.status(500).json({ error: 'Failed to send reset email' });
  }
});

// POST /api/users/reset-password
router.post('/reset-password', async (req, res) => {
  var token = req.body.token;
  var password = req.body.password;
  if (!token || !password) return res.status(400).json({ error: 'Token and password required' });
  if (password.length < 8) return res.status(400).json({ error: 'Password must be at least 8 characters'

  try {
    var result = await pool.query(
      `SELECT prt.*, u.email FROM password_reset_tokens prt
       JOIN users u ON prt.user_id = u.id
       WHERE prt.token = $1 AND prt.used_at IS NULL AND prt.expires_at > NOW()`,
      [token]
    );

    if (!result.rows.length) {
      return res.status(400).json({ error: 'Reset link is invalid or has expired' });
    }

    var resetToken = result.rows[0];
    var hash = await bcrypt.hash(password, 12);

    await pool.query('UPDATE users SET password_hash = $1 WHERE id = $2', [hash, resetToken.user_id]);
    await pool.query('UPDATE password_reset_tokens SET used_at = NOW() WHERE id = $1', [resetToken.id]);

    res.json({ success: true });
  } catch (err) {
    console.error('Reset password error:', err);
    res.status(500).json({ error: 'Failed to reset password' });
  }
});

module.exports = router;
```

```
========================================================================
FILE: server.js
========================================================================
```

```
const express = require('express');
```

```
const cors = require('cors');
const path = require('path');
const session = require('express-session');
const pgSession = require('connect-pg-simple')(session);
const { Pool } = require('pg');
const { execFile } = require('child_process');
require('dotenv').config();

const app = express();
app.set('trust proxy', 1);
const PORT = process.env.PORT || 3000;
const pool = new Pool({ connectionString: process.env.DATABASE_URL });

if (process.env.NODE_ENV === 'production' && !process.env.SESSION_SECRET) {
  console.error('FATAL: SESSION_SECRET must be set in production');
  process.exit(1);
}

app.use(cors({
  origin: process.env.ALLOWED_ORIGIN || 'http://localhost:3000',
  credentials: true
}));
app.use(express.json({ limit: '50mb' }));

app.use(session({
  store: new pgSession({ pool, tableName: 'sessions', createTableIfMissing: true }),
  secret: process.env.SESSION_SECRET || 'slater-dev-secret',
  resave: false,
  saveUninitialized: false,
  cookie: {
    maxAge: 24 * 60 * 60 * 1000,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict'
  }
}));

pool.query('SELECT 1').then(async () => {
  console.log('Database connected');
  await pool.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS contacts JSONB DEFAULT '{}'`);
  await pool.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS logo TEXT`);
  console.log('Migrations complete');
}).catch(err => {
  console.error('Database connection failed:', err.message);
});

app.use((req, res, next) => {
  console.log(req.method, req.path);
  next();
});

function requireAuth(req, res, next) {
  if (req.session && req.session.userId) return next();
  res.status(401).json({ error: 'Not logged in' });
}

app.use('/api/contacts', requireAuth, require('./routes/contacts'));
app.use('/api/users/me/contacts', requireAuth, require('./routes/contacts'));
app.use('/api/users', require('./routes/users'));

app.use('/api/agencies', requireAuth, require('./routes/agencies'));
app.use('/api/projects', requireAuth, require('./routes/projects'));
app.use('/api/receipts', requireAuth, require('./routes/receipts'));
app.use('/api/ocr', requireAuth, require('./routes/ocr'));
app.use('/api/licenses', require('./routes/licenses'));

app.get('/health', (req, res) => {
```

```
  res.json({ status: 'ok', version: '0.1.0' });
});

// --- Admin auth ---

function requireAdmin(req, res, next) {
  if (req.session && req.session.isAdmin) return next();
  res.redirect('/admin/login');
}

app.get('/admin/login', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'admin-login.html'));
});

app.post('/admin/login', (req, res) => {
  if (req.body.secret === process.env.ADMIN_SECRET) {
    req.session.isAdmin = true;
    res.json({ success: true });
  } else {
    res.status(401).json({ error: 'Invalid admin password' });
  }
});

app.get('/admin', requireAdmin, (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'admin.html'));
});

app.get('/admin/logout', (req, res) => {
  req.session.isAdmin = false;
  res.redirect('/admin/login');
});

// --- Admin API ---

app.get('/api/admin/stats', requireAdmin, async (req, res) => {
  try {
    const [users, licenses, devices, projects] = await Promise.all([
      pool.query('SELECT COUNT(*) FROM users'),
      pool.query('SELECT COUNT(*) FROM licenses'),
      pool.query('SELECT COUNT(*) FROM license_devices'),
      pool.query('SELECT COUNT(*) FROM projects'),
    ]);
    res.json({
      users: parseInt(users.rows[0].count),
      licenses: parseInt(licenses.rows[0].count),
      devices: parseInt(devices.rows[0].count),
      projects: parseInt(projects.rows[0].count),
    });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

app.get('/api/admin/users', requireAdmin, async (req, res) => {
  try {
    const result = await pool.query(`
      SELECT u.id, u.name, u.email, u.created_at,
             l.key as license_key, l.plan, l.expires_at,
             COUNT(ld.id) as device_count
      FROM users u
      LEFT JOIN licenses l ON u.license_id = l.id
      LEFT JOIN license_devices ld ON l.id = ld.license_id
      GROUP BY u.id, l.key, l.plan, l.expires_at
      ORDER BY u.created_at DESC
    `);
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});
```

```
10866
10867 app.get('/api/admin/licenses', requireAdmin, async (req, res) => {
10868   try {
10869     const result = await pool.query(`
10870       SELECT l.*, COUNT(ld.id) as device_count,
10871              u.email as user_email, u.name as user_name
10872       FROM licenses l
10873       LEFT JOIN license_devices ld ON l.id = ld.license_id
10874       LEFT JOIN users u ON u.license_id = l.id
10875       GROUP BY l.id, u.email, u.name
10876       ORDER BY l.created_at DESC
10877     `);
10878     res.json(result.rows);
10879   } catch (err) { res.status(500).json({ error: err.message }); }
10880 });
10881
10882 app.post('/api/admin/licenses/generate', requireAdmin, async (req, res) => {
10883   try {
10884     const crypto = require('crypto');
10885     var count = req.body.count || 1;
10886     var plan = req.body.plan || 'beta';
10887     var max_devices = req.body.max_devices || 3;
10888     var expires_at = req.body.expires_at;
10889     var generateKey = function() {
10890       var seg = function() { return crypto.randomBytes(2).toString('hex').toUpperCase(); };
10891       return 'SLTR-' + seg() + seg() + '-' + seg() + seg() + '-' + seg() + seg();
10892     };
10893     var keys = [];
10894     for (var i = 0; i < Math.min(count, 50); i++) {
10895       var key = generateKey();
10896       await pool.query(
10897         'INSERT INTO licenses (key, plan, max_devices, expires_at) VALUES ($1, $2, $3, $4)',
10898         [key, plan, max_devices, expires_at || null]
10899       );
10900       keys.push(key);
10901     }
10902     res.json({ keys: keys });
10903   } catch (err) { res.status(500).json({ error: err.message }); }
10904 });
10905
10906 app.delete('/api/admin/licenses/:id', requireAdmin, async (req, res) => {
10907   try {
10908     await pool.query('DELETE FROM licenses WHERE id = $1', [req.params.id]);
10909     res.json({ success: true });
10910   } catch (err) { res.status(500).json({ error: err.message }); }
10911 });
10912
10913 app.delete('/api/admin/users/:id', requireAdmin, async (req, res) => {
10914   try {
10915     await pool.query('DELETE FROM users WHERE id = $1', [req.params.id]);
10916     res.json({ success: true });
10917   } catch (err) { res.status(500).json({ error: err.message }); }
10918 });
10919
10920 app.post('/api/admin/sync-prod-db', requireAdmin, (req, res) => {
10921   if (process.env.NODE_ENV === 'production') {
10922     return res.status(403).json({ error: 'Not available in production' });
10923   }
10924   const scriptPath = path.join(__dirname, 'scripts', 'sync-prod-db.sh');
10925   execFile('bash', [scriptPath], { timeout: 120000 }, (err, stdout, stderr) => {
10926     if (err) {
10927       console.error('sync-prod-db error:', stderr);
10928       return res.status(500).json({ error: stderr || err.message });
10929     }
10930     res.json({ ok: true, log: stdout });
10931   });
```

```
});

// --- End admin ---

app.get('/login', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'login.html'));
});

app.get('/reset-password', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'reset-password.html'));
});

app.get('/', (req, res, next) => {
  if (!req.session || !req.session.userId) return res.redirect('/login');
  next();
});

app.use(express.static(path.join(__dirname, 'public')));

app.listen(PORT, () => {
  console.log(`Slater server running on http://localhost:${PORT}`);
});
```


```
======================================================================
COPYRIGHT NOTICE (appended per deposit rules -- public/index.html)
======================================================================
        <div class="sb" id="status-bar"></div>

        <div class="slater-footer">
          <strong>Slater</strong>  &middot;  &copy; <script>document.write(new Date().getFullY
          <div style="margin-top:8px;display:flex;align-items:center;justify-content:center;gap:12px">
            <button onclick="exportLibrary()" style="background:none;border:none;cursor:pointer;font-siz
            <span style="color:var(--border)">&middot;</span>
```